Repositorio del proyecto: https://github.com/donmatthiuz/RL/tree/proyecto

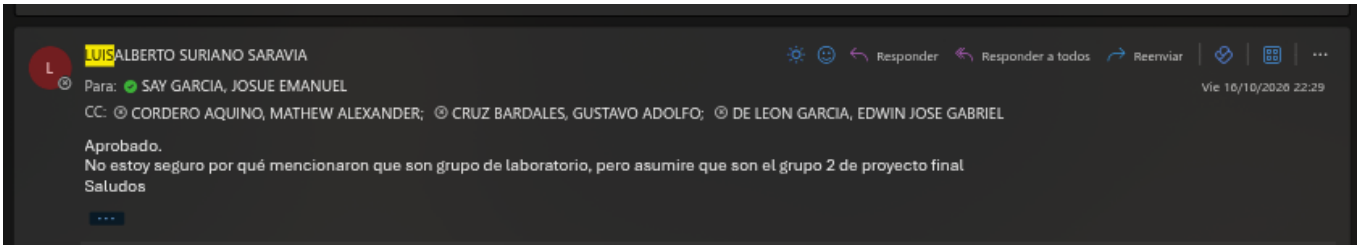
EVIDENCIA DOCUMENTAL

Constancia de entrega y aprobación

Propuesta de proyecto: Aprendizaje por Refuerzo en Polymarket

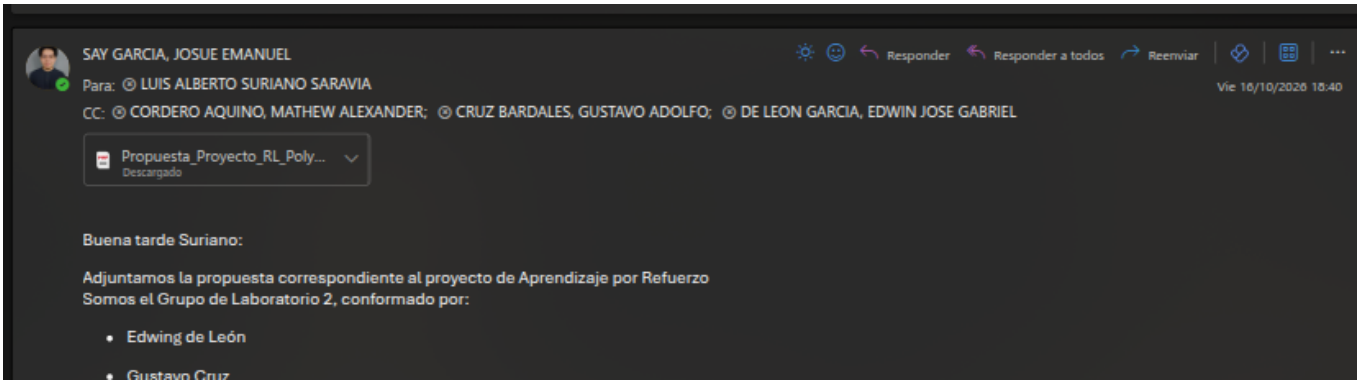
Se incorporan al inicio las capturas aportadas como evidencia de entrega oportuna de la propuesta original. En ellas constan el envío del **16 de octubre de 2026 a las 18:40** y la aprobación del profesor ese mismo día a las **22:29**.

1. Aprobación del profesor



Luis Alberto Suriano Saravia · 16/10/2026, 22:29. El mensaje confirma: «Aprobado». También indica que asume que el equipo corresponde al **grupo 2 del proyecto final**.

2. Envío de la propuesta



Jose Emanuel Say García a Luis Alberto Suriano Saravia · 16/10/2026, 18:40. La captura muestra el correo de envío, el archivo adjunto y la referencia al proyecto de Aprendizaje por Refuerzo.

> **Actualización de la propuesta: 23 de octubre de 2026.**
> Las páginas siguientes desarrollan la propuesta original e incorporan los cambios de documentación y código registrados en el repositorio durante esta fecha. Las capturas se conservan sin modificaciones.

PROPUESTA ACTUALIZADA · 23 DE OCTUBRE DE 2026

PolyRL: aprendizaje por refuerzo en Polymarket

Datos históricos, decisiones secuenciales y operaciones con saldo ficticio.

Idea y objetivo

Construir un agente que observe un mercado binario histórico y aprenda cuándo **comprar YES, comprar NO, vender o esperar**. El objetivo es comprobar si su política supera a estrategias simples en mercados que no se utilizaron para entrenarlo.

Polymarket funciona como fuente de datos. El entrenamiento y la evaluación se realizarán en un simulador con **saldo ficticio**. El proyecto integra Deep Reinforcement Learning, datos reales, modelado de un entorno y evaluación estadística.

Cómo funcionará

- **1. Datos:** seleccionar mercados resueltos de BTC, descargar sus precios y alinearlos con el precio spot. ETH queda como extensión si el tiempo lo permite.
- **2. Simulación:** reproducir cada mercado como un episodio, con una posición a la vez, costos configurables y liquidación al conocerse el resultado.
- **3. Aprendizaje:** entrenar PPO y DQN interactuando con el simulador; entrenar la variante offline con un conjunto fijo de transiciones.
- **4. Comparación:** ejecutar todas las políticas sobre los mismos mercados de evaluación y medir retorno, riesgo, operaciones y costos.

Técnicas que se utilizarán

Técnica	Aplicación prevista
PPO	Agente principal; optimiza la política mediante una arquitectura actor-crítico.
DQN	Agente de comparación; estima el valor de las cuatro acciones discretas.
Offline RL	Discrete CQL; Behavior Cloning como referencia, ambos sobre transiciones fijas.
Backtesting y recompensa	Reproducción histórica y comparación de recompensas basadas en PnL, riesgo y resultado final.

> **Precisión conceptual incorporada:** usar precios históricos en un simulador no convierte por sí solo a PPO o DQN en Offline RL. Si el agente interactúa con el entorno, el entrenamiento es online. La variante offline aprende de transiciones ya recolectadas.

Resultado esperado

Una demo local en Streamlit que reproduzca un mercado paso a paso y muestre la acción, la posición y el saldo, junto con curvas de capital y una tabla comparativa frente a estrategias base. El informe explicará también los resultados negativos y las limitaciones.

DISEÑO TÉCNICO PREVISTO

Datos, entorno y recompensa

La arquitectura se organiza por módulos y contratos para permitir el trabajo del equipo.

Módulo	Responsabilidad y salida
Datos y features	Ingesta, limpieza y alineación temporal. Salidas previstas: markets.parquet, steps.parquet y manifest.json con hashes.
Entorno	PolymarketEnv compatible con Gymnasium; cartera, ejecución, costos, liquidación y componentes de recompensa.
Agentes	Interfaz común Policy para baselines, PPO, DQN y CQL/BC; checkpoints y configuración por corrida.
Evaluación y demo	Backtest común, métricas y trayectorias. La demo leerá resultados guardados para reproducir las decisiones.

Datos y prevención de fuga de información

La integración ya incorporada consulta metadatos, precios e historial mediante el paquete **rl_polymarket**. El pipeline completo añadirá caché, filtros de calidad y precio spot de BTC/ETH. La familia inicial propuesta es *Bitcoin Up or Down*; su selección definitiva depende de validar la granularidad de los mercados cerrados.

El conjunto se dividirá cronológicamente **por mercado**: aproximadamente 70 % para train, 15 % para validación y 15 % para test. Se usará solo información disponible hasta el instante observado, relleno hacia adelante y normalización ajustada únicamente en train.

Formulación del entorno

- **Estado:** precios YES/NO, cambios y volatilidad recientes, precio spot, tiempo restante, lado y tamaño de la posición, PnL no realizado y saldo normalizado.
- **Acciones:** WAIT, BUY_YES, BUY_NO y SELL. Una sola posición abierta y tamaño fijo configurable; sin apalancamiento y con saldo no negativo.
- **Transición:** avance al siguiente dato histórico. Se modelarán spread, slippage y comisión; las acciones inválidas se convertirán en WAIT y se penalizarán.
- **Final del episodio:** resolución del mercado y liquidación de la posición a 1 o 0 según el resultado del token mantenido.

Función de recompensa y variantes

La base será el cambio del valor total de la cartera normalizado por el capital inicial. La variante principal añadirá una penalización por aumento del drawdown. Los costos se reflejarán en el valor de la cartera; se podrán estudiar penalizaciones adicionales para acciones inválidas y tiempo de exposición.

> **Variantes para comparar:** *pnl* (cambio de valor por paso), *pnl_risk* (PnL y riesgo) y *terminal* (resultado final). Con *pnl*, la suma de recompensas debe coincidir con el PnL final dividido por el capital inicial.

ORGANIZACIÓN Y PLAN DE TRABAJO

Equipo, fases y entregables

Cinco roles documentados; asignaciones tomadas de docs/03_roles.md.

Rol / integrante	Responsabilidad principal	Respaldo
R1 · Por asignar Líder técnico e integrador	Integración, contratos, CLI, herramientas, CI/CD y coherencia de la documentación.	R4
R2 · Mathew Ingeniería de datos	APIs, dataset, features, split temporal y análisis exploratorio.	R5
R3 · Gustavo Entorno y recompensa	Simulador, cartera, costos, liquidación, invariantes y ablación de recompensa.	R4
R4 · Por asignar Agentes RL	Baselines, PPO, DQN, dataset offline, CQL/BC y entrenamiento reproducible.	R3
R5 · Por asignar Evaluación y demo	Backtesting, métricas, estadística, Streamlit, informe y presentación.	R2

Fases e hitos previstos

Fase	Hito verificable de cierre
F0 · Preparación	Herramientas y CI operativas; validar las APIs, acordar el MDP, el protocolo y las asignaciones pendientes.
F1 · Datos y entorno	Dataset v1, entorno validado y un baseline ejecutando episodios completos sobre datos reales.
F2 · Agentes online	PPO y DQN entrenados; prueba en entorno sintético y primera comparación en validación.
F3 · Offline y ajuste	CQL/BC, ablación de recompensa, modelos finales con cinco semillas y demo inicial.
F4 · Entrega final	Evaluación final en test, demo comparativa, informe, presentación y artefactos reproducibles.

Las fases expresan el orden del trabajo; no fijan fechas. Las tareas se clasifican en P0 (esenciales), P1 (de mejora) y P2 (opcionales). Si el tiempo es limitado, se recortarán primero las P2, incluida la extensión a ETH y la demo pública.

Colaboración y criterio de finalización

El trabajo se concentrará en la rama **proyecto**, con commits pequeños, sincronización antes de subir y pruebas por cada cambio de lógica. Cada rol mantiene sus módulos, documenta sus decisiones y contribuye al informe. Una tarea se dará por terminada cuando cumpla su aceptación, tenga pruebas y documentación y esté integrada con CI en verde.

METODOLOGÍA Y CRITERIOS DE ÉXITO

Evaluación y calidad

El protocolo busca comprobar qué aprende el agente y bajo qué supuestos funciona.

Comparación de estrategias

Los agentes se compararán con cinco estrategias base: **aleatoria, no operar, favorito, momentum y reversión a la media**. Todas utilizarán el mismo entorno, costos y mercados. Se seleccionarán modelos con validación; el test se reservará para la evaluación final, una vez congelados modelos e hiperparámetros.

- **Métricas:** retorno medio por mercado, retorno acumulado, Sharpe por episodio, drawdown máximo, win rate, número de operaciones, costos y porcentaje de acciones inválidas.
- **Repeticiones:** al menos cinco semillas para las configuraciones finales; media y dispersión entre semillas e intervalos de confianza del 95 % por bootstrap.
- **Análisis:** comparación pareada sobre los mismos mercados, ablación de la recompensa y, si el tiempo lo permite, sensibilidad a costos y ejecución al siguiente paso.

Hipótesis y entregables evaluables

Se estudiará si PPO supera al mejor baseline fuera de muestra, si penalizar drawdown reduce el riesgo y si CQL iguala o supera a las políticas que generaron sus datos. **No superar los baselines también será un resultado válido** si la evaluación es rigurosa y la conclusión queda sustentada.

Entregable	Verificación prevista
Dataset y entorno	Reconstrucción con el mismo hash; check_env de Gymnasium; conservación de cartera, saldo no negativo y ausencia de datos futuros.
Agentes y comparación	Checkpoints y curvas de aprendizaje para PPO, DQN y la variante offline; tabla en test con cinco semillas e IC del 95 %.
Demo e informe	Reproducción paso a paso, comparación visual y reporte de metodología, resultados, ablación y limitaciones.
Calidad y reproducibilidad	CI en proyecto; cobertura mínima objetivo de 70 % general y 90 % en módulos críticos. Configuración, semilla, dataset y commit por corrida.

Pruebas previstas

Las pruebas unitarias usarán respuestas simuladas y semillas controladas. Se verificarán splits sin intersección, features sin información futura, liquidación, costos y acciones inválidas. Se añadirán pruebas de propiedades, integración de extremo a extremo y entrenamientos breves; las pruebas de red y los entrenamientos largos se ejecutarán por separado.

Riesgos y límites del diseño

Los principales riesgos son datos históricos poco granulares, fuga de información, sobreajuste, recompensa mal calibrada e integración tardía. Se mitigarán validando los datos primero, reservando test, usando fixtures y probando los módulos desde el inicio. El simulador asumirá ejecución price-taker y costos configurables; aproximará **p(NO) como 1 - p(YES)**. No reconstruirá el libro histórico completo. Estos supuestos se explicarán al interpretar los resultados.

CAMBIOS REGISTRADOS · 23 DE OCTUBRE DE 2026

Avances y estado actual

Resumen basado en el historial local de Git, los archivos del proyecto y las pruebas ejecutadas.

Hora / commit	Cambio realizado
16:53 · 9e2c886	README con el índice, el objetivo del proyecto y la ruta de lectura de la documentación.
16:56 · 2271f36	Siete documentos de planificación: idea, arquitectura, roles, tareas, testing, flujo de trabajo y riesgos/decisiones.
18:39 · 08e7188	Paquete rl_polymarket, configuración, CLI de consulta, guía de integración, plantilla de variables y pruebas. Actualización de la asignación de R2 a Mathew.

Horas de los commits con desfase UTC−06:00, correspondiente a Guatemala.

Implementado y comprobado

- **Base instalable:** Python 3.11 o posterior, pyproject.toml, dependencia fijada polymarket-client==0.14.0 y comando **rl-polymarket**.
- **Consultas:** mercados y detalle por slug, libro, precio de compra/venta, punto medio e historial paginado. El paquete selecciona identificadores de resultado v1/v2 y conserva precisión decimal en la salida JSON.
- **Cuenta opcional:** configuración con variables de entorno y .env; acceso autenticado explícito para consultar cuenta y órdenes abiertas. Los comandos implementados consultan información y no envían ni cancelan órdenes.

> **Verificación local:** `venv/bin/python -m pytest -q`
> **Resultado: 22 pruebas aprobadas.** Cubren configuración, autenticación simulada, paginación, precisión de precios, historial, errores y cierre de conexiones. El transporte HTTP está simulado; este resultado valida la integración local.

Diferencia entre la base actual y la arquitectura prevista

La base actual utiliza **src/rl_polymarket** y una CLI con **argparse**. La documentación describe como destino el paquete **polyrl**, la configuración YAML y los comandos de datos, entrenamiento y evaluación. Esa integración sigue pendiente, junto con el dataset procesado, Gymnasium, las funciones de recompensa, los agentes entrenados, el backtesting, la demo y los workflows de CI/CD. A esta fecha no se reportan resultados experimentales ni hipótesis confirmadas.

Documentos que sustentan esta actualización

Propuesta original: *Propuesta_Proyecto_RL_Polymarket_final (2).pdf*. Evidencia: *image copy.png* e *image.png*. Desarrollo técnico: **README.md**, **docs/01_idea_general.md** a **docs/07_riesgos_y_decisiones.md**, **docs/polymarket.md**, **pyproject.toml**, **src/rl_polymarket/** y **tests/test_polymarket.py**.

> **Repositorio del proyecto · rama proyecto**
> https://github.com/donmatthiuz/RL/tree/proyecto